



Algorithmic Toolkit for Linearization of S-Boxes

Alex Biryukov^{1,2}, Philip Tureček², and Aleksei Udovenko²(✉)

¹ DCS, University of Luxembourg, Esch-sur-Alzette, Luxembourg
`alex.biryukov@uni.lu`

² SnT, University of Luxembourg, Esch-sur-Alzette, Luxembourg
`philip.turecek@uni.lu`, `aleksei@affine.group`

Abstract. Linearization is a cryptanalysis technique in which a nonlinear function (an S-box) is represented by an affine mapping on a certain subset of inputs. Its variants were applied to analyze Keccak, LowMC, RAIN and AIM. In these primitives, the S-boxes are either very small (up to 5 bits) or are very specific monomial functions over a binary field. Linearization of arbitrary S-boxes was never practically explored due to the lack of theoretic, algorithmic, and cryptanalytic understanding.

For the first time, we develop an algorithmic toolkit which allows one to compute strong linearizations of S-boxes, when they exist. For up to $n = 8$ bits, our algorithms are able to find provably the best possible approximations, while for larger S-boxes it is feasible to obtain good approximations together with meaningful upper bounds. We apply our algorithms to a variety of S-boxes from existing primitives, to monomial functions, to so-called APN functions, and to 16-bit Super-Sboxes. We obtain interesting results raising many new open questions and open up new research directions, as well as a foundation for developing cryptanalytic attacks.

To advance the cryptanalytic utility of linearization, we study and solve the problem of covering an S-box with multiple approximations. As an application, we derive a generic linearization approach for the CICO problem (constrained-input-constrained-output) over SPN-based permutations (Substitution-Permutation Networks) with general linear layers. This is the first such general cryptanalysis based on the existence of a strong linearization of the S-box.

Keywords: Boolean functions · Linearization · Vectorial linearity · S-boxes · CICO

Work funded by Luxembourg's FNR projects CryptoFin (C22/IS/17415825) and PQseal (C24/IS/18978392). We thank Luca Bonamino for Rust implementations of the greedy extension algorithm and the greedy reduction variant for the AES Super-Sbox. We also thank the anonymous reviewers for their comments.

© International Association for Cryptologic Research 2026
J. Daemen and E. Thomé (Eds.): EUROCRYPT 2026, LNCS 16546, pp. 243–272, 2026.
https://doi.org/10.1007/978-3-032-25333-0_9

1 Introduction

In symmetric-key cryptography, S-boxes are key components of encryption algorithms such as block ciphers or those that involve permutations. S-boxes are nonlinear functions defined over a small domain and can be described by a lookup table. These functions are typically the only source of nonlinearity in those encryption primitives, and thus it is crucial to understand their security properties such as resistance to various attacks.

The first and main security properties of S-boxes that were studied are the differential uniformity and the (non)linearity [34, 35], quantifying resistance of the S-box against the differential [7] and linear cryptanalysis [29, 31] respectively. Since then, many other attacks and corresponding properties have been studied, such as algebraic degree, correlation/algebraic immunity, boomerang and rectangle uniformity, nonlinear invariants, algebraic degrees of polynomial relations.

A much less studied concept is the *vectorial* linearity of an S-box, which characterizes the size of the best possible linear approximation of the full function (i.e., of all output bits simultaneously), the process called *linearization*. First defined in [15], this notion was only theoretically studied in a few subsequent works [12, 17, 28, 32, 40]. More formally, *vectorial nonlinearity* is defined as the Hamming distance from the given vectorial Boolean function (an S-box) to the set of affine mappings with the same signature. Equivalently, *vectorial linearity* is equal to the maximum size of a subset of the domain of the S-box, on which it can be described by an affine map, called *affine approximation*.

In cryptanalysis, linearization is particularly useful in low-data attacks, especially in the setting with only one known plaintext-ciphertext pair, where traditional statistical cryptanalysis techniques such as differential, linear, and integral cryptanalysis are not applicable. This setting became practically relevant due to post-quantum signature schemes based on symmetric-key primitives, using the so-called multiparty-computation-in-the-head paradigm and its successors. Examples include Picnic/LowMC [43], AIMer [26], Banquet [3]. AIMer was recently selected as the winner of the Korean Post-Quantum Cryptography Competition (KpqC), and the AES-based Banquet is a current candidate of the ongoing NIST Additional Post-Quantum Signatures Competition. The designers of the 64-bit permutation Alzette [4] also studied the linearization of the ARX-based component (Addition-Rotation-Xor) based on solving a system of quadratic equations.

There are several cryptanalysis results using linearization, in particular, low-data algebraic meet-in-the-middle attacks on LowMC [1], RAIN and AIM/AIMER [44]. Note that LowMC has a 3-bit S-box, RAIN employs the binary finite field inversion, and AIM uses monomials with exponents of the form $2^e - 1$ for some e . The authors of [44] use a decomposition of the inverse function as $x^{2^n-2} = (x^{2^{n/2}+1})^{2^{n/2}-2} \cdot x^{2^{n/2}}$ which is only possible for an even field dimension n , and they suggest that using an odd n would prevent their attack. It is thus a very natural question of what is the best linearization of the inverse function on odd numbers of bits. More generally, understanding linearization possibilities for

monomial functions is crucially relevant for arithmetization-oriented (AO) primitives. As a further example, the authors of [44] linearized monomial functions of AIM-III with only $45/2^{192}$ elements, which seems very low for such a large field (although here a linearization must hold for two monomial functions at the same time). Linearization is also a key component in the cryptanalysis of hash functions such as Keccak [23, 39] or AES-based hashing [16], where sometimes an additional constraint requires that the restricted domain is an affine space rather than an arbitrary set. In a nutshell, the attacker can trade some available degrees of freedom in order to propagate through the S-box in a way that the input and output variables are linearly related or constant.

Finally, we note that the constrained-input-constrained-output (CICO) problem for permutations, underlying the security of sponge-based hash functions and other primitives, is very similar in spirit to the single-plaintext setting, and linearization is also a very useful tool, as we will show in Sect. 7. The CICO setting is also the main one used for cryptanalysis of AO hash functions.

In this work, our aim is to advance the study of vectorial linearity by developing algorithms for finding vectorial affine approximations of S-boxes in practice, as well as a theoretical framework to study such approximations in terms of the difference or correlation tables of the S-box.

Our Contribution. Our main contribution is the development of powerful algorithms for finding strong linearizations of S-boxes in practice. For up to dimension 8, we are able to identify *provably optimal* affine approximations, and we offer high-quality heuristic results for larger sizes.

The algorithms¹ are summarized in Table 1. The “extension” algorithms are closely related to the difference distribution table of the S-box (DDT), while the “reduction” algorithms are closely related to the linear approximation table of the S-box (LAT). We develop natural greedy and exhaustive versions of each flavor for completeness, although in practice, the most useful ones are Greedy-Extension and Exhaustive-Reduction. As a by-product, our technique allows one to slightly improve the general lower bound on vectorial linearity from $n+1$ (due to Carlet [12]) to $n+2$, which was posed as an open problem at the NSUCRYPTO 2021 [18] international cryptographic olympiad.

We apply the algorithms to a variety of S-boxes: monomial functions on $\mathbb{F}_{2^n}$, S-boxes designed for block or stream ciphers, APN functions, and several 16-bit Super-Sboxes. We find that the 8-bit S-box of SKINNY has an affine approximation valid on 47 out of 256 inputs, while for most other S-boxes this value is in the range of 18–30, closer to 20–22 for random 8-bit S-boxes. We also show that the AES S-box has no better approximations than the one based on the decomposition from [44], with 18 points. We observe a wide range of vectorial linearity² among 16-bit Super-Sboxes, ranging from 49 (MISTY-FI function [30]),

¹ All the implementations and results from the paper will be made publicly available at [9] and https://github.com/cryptolu/sbox_linearization.

² For 16-bit S-boxes our results are only heuristic but we believe they are close to optimal.

and around 110 (LED [24]) to around 1100 (SKINNY64 [5]) and 4096 (Midori64 [2]). We investigate the two latter cases and uncover the reasons for such a weakness. We also ran a simplified variant of the Greedy-Reduction algorithm on the 32-bit AES Super-Sbox (with the zero key) and obtained an approximation of size 60. We leave it as a challenge to improve this difficult case.

Finally, we describe new methods for covering an S-box with several approximations, which is necessary to increase the success rate of linearization attacks on primitives. We then apply the method to a generic CICO problem over SPN with general linear layers and derive bounds on the vectorial linearity of S-boxes that allow for linearization attacks for varying numbers of rounds.

Table 1. Summary of our algorithms applied to an S-box $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. For exhaustive variants, the complexity estimates assume that $g = \text{VecLin}_S \geq 2^{n/2}$ and $k = \lfloor \frac{n}{2} \rfloor + 2$. Optimizations for special cases are not included.

Algorithm	Ref.	Nature	Complexity (Per sample)	Complexity (Total)
Brute-force	Sect. 3.1	Exhaustive	$\mathcal{O}(2^n)$	$\mathcal{O}(\frac{2^{n(n+2)}}{(n+1)!})$
Sampling	Sect. 3.1	Heuristic	$\mathcal{O}(2^n)$	$\mathcal{O}(2^{(n+1)(n+1-\log_2 g)})$
Extension-Exhaustive	Full ver.	Exhaustive	–	$\mathcal{O}(\frac{2^{nk}}{k!})$
Extension-Greedy	Sect. 4.3	Heuristic	$\mathcal{O}(2^n)$	–
Reduction-Exhaustive	Sect. 5.1	Exhaustive	–	$\mathcal{O}(2^{(n+1)(k+0.5)})$
Reduction-Greedy	Sect. 5.2	Heuristic	$\mathcal{O}(2^{n+m})$	–

2 Preliminaries

The *weight* of a map $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, denoted $\text{wt}(S)$, is defined as the number of inputs $x \in \mathbb{F}_2^n$ for which $S(x) \neq 0$. The inner product of two vectors x, y of length n is denoted by $\langle x, y \rangle = \sum_{i=1}^n x_i y_i$.

Linear Approximations. The *Walsh transform* of a Boolean function $s: \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ is the map $W_s: \mathbb{F}_2^n \rightarrow \mathbb{Z}$ given by $W_s(u) = \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle u, x \rangle + s(x)}$. The *Walsh transform* (also known as the *linear approximation table* – LAT) of a vectorial Boolean function $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ is the map $W_S: \mathbb{F}_2^n \times \mathbb{F}_2^m \rightarrow \mathbb{Z}$ given by

$$W_S(u, v) = \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle u, x \rangle + \langle v, S(x) \rangle}.$$

The (Boolean) *linearity* of a (vectorial) function S , denoted Lin_S , is the maximum absolute value of W_S excluding $W_S(0, 0) = 2^n$. The Walsh transform can be computed efficiently in time $\mathcal{O}(n2^n)$ for a Boolean function (implying $\mathcal{O}((n+m)2^{n+m})$ for a vectorial Boolean function) using the so-called Fast Walsh-Hadamard Transform (FWHT), which is essentially the multi-dimensional variant of the Fast Fourier Transform.

Algebraic Degree. Let $f: \mathbb{F}_2^n \rightarrow \mathbb{F}_2$. The *algebraic normal form* (ANF) of the function f is the unique representation of f as a multivariate polynomial in $\mathbb{F}_2[x_1, \dots, x_n]/\langle x_1^2 - x_1, \dots, x_n^2 - x_n \rangle$. For a vectorial function $F: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, the same representation may be achieved in each of its m coordinates. The *algebraic degree* $\deg F$ of $F = (f_1, \dots, f_m)$ is defined as the maximum degree among all degrees $\deg(f_i)$ for $i \in \{1, \dots, m\}$. For $f = 0$, the degree is set to $-\infty$.

Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. For any fixed $\Delta \in \mathbb{F}_2^n$, define the *derivative* of S in the direction of Δ as $\mathbf{D}_\Delta S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, $x \mapsto S(x + \Delta) + S(x)$. We allow the trivial derivative $\mathbf{D}_0(S)$ which is the zero function. For any nonzero function S , taking a derivative strictly decreases the algebraic degree: $\deg \mathbf{D}_\Delta S \leq \deg S - 1$.

The finite field $\mathbb{F}_{2^n}$ can be represented by the polynomials in $\mathbb{F}_2[x]/\langle Q(x) \rangle$, where $Q(x) \in \mathbb{F}_2[x]$ is an irreducible polynomial of degree n . Every function $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ has a unique equivalent representation as a univariate polynomial $P(x)$ in $\mathbb{F}_{2^n}[x]/\langle x^{2^n} - x \rangle$, i.e., of degree at most $2^n - 1$. The algebraic degree of S is equal to the maximum Hamming weight of exponents present in $P(x)$.

Affine Subspaces. An *affine subspace* A of a vector space V is a coset of a linear subspace U of V : $A = U + a$ for $U \subseteq V$ and a constant $a \in V$. The *dimension* of A is defined as the dimension of U . Two affine subspaces are said to be *parallel* if they are cosets of the same linear subspace.

An *affine combination* of vectors $a_0, \dots, a_n$ is a linear combination whose coefficients sum to 1. The set of all affine combinations of $a_0, \dots, a_n$ is called the *affine span* of $a_0, \dots, a_n$, also written as $\langle a_0, \dots, a_n \rangle_{\text{aff}}$; it is an affine subspace A . We say that these vectors are *affinely independent* if the vectors $a_1 - a_0, \dots, a_n - a_0$ are linearly independent, in which case these vectors form an *affine basis* of A . An affine basis of a d -dimensional affine subspace consists of $d + 1$ vectors.

The number of ordered affine bases of $\mathbb{F}_2^n$ is $\mathcal{N}_n := 2^n \prod_{i=0}^{n-1} (2^n - 2^i) \approx 0.29 \cdot 2^{n(n+1)}$ whereas the number of unordered affine bases of $\mathbb{F}_2^n$ is $\mathcal{N}_n / (n + 1)!$.

An *affine map* F is a map of the form $F(x) = L(x) + a$ for a linear map L and a constant vector a . Affine maps preserve affine combinations:

$$F(\lambda_0 a_0 + \dots + \lambda_n a_n) = \lambda_0 F(a_0) + \dots + \lambda_n F(a_n)$$

for all vectors $a_0, \dots, a_n$ and coefficients $\lambda_0 + \dots + \lambda_n = 1$.

Let $X = \{x_0, \dots, x_n\}$ be a set of vectors. We define the *affine rank* of X as the maximum number of affinely independent vectors in it. This coincides with the dimension of the (linear) span of the *extended* vectors $(1, x_0), \dots, (1, x_n)$.

Zero-Sum Sets

Definition 1. Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. The difference distribution table $DDT_S: \mathbb{F}_2^n \times \mathbb{F}_2^m \rightarrow \mathbb{Z}_{\geq 0}$ of S is given by

$$DDT_S(\alpha, \beta) = |\{x \in \mathbb{F}_2^n \mid S(x) + S(x + \alpha) = \beta\}|.$$

The function S is said to be differentially δ_S -uniform where δ_S is the maximum value of DDT_S excluding $DDT_S(0, 0) = 2^n$.

In our extension algorithms, we will use a variant of the DDT which records the actual solutions instead of their number.

Definition 2 (Zero-sums, cf. [13, Def. 1]). Let k be a positive even integer and $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ an arbitrary map. We say that k pairwise distinct points $p_1, \dots, p_k \in \mathbb{F}_2^n$ such that $p_1 + \dots + p_k = 0$ and $S(p_1) + \dots + S(p_k) = 0$ form a k -zero-sum of S . We call the dimension of the affine span of $\{p_1, \dots, p_k\}$ its input dimension and the dimension of the affine span of $\{(p_1, S(p_1)), \dots, (p_k, S(p_k))\}$ its full dimension.

The smallest (even) k for which a k -zero-sum may exist is 4, because we consider distinct inputs and two distinct elements in characteristic 2 cannot sum to zero. The proof of the following proposition is given in the full version of the paper.

Proposition 1. Let k be a positive even integer and $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. For every k -zero-sum of S formed by pairwise distinct elements $p_1, \dots, p_k \in \mathbb{F}_2^n$, we have

$$\lceil \log_2(k) \rceil \leq \text{input dimension} \leq \text{full dimension} \leq k - 2.$$

For $k \in \{4, 6\}$, every k -zero-sum has input dimension $k - 2$, and thus, also full dimension $k - 2$.

Definition 3 ([36]). A map $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ is called almost perfect nonlinear (APN) if S has no 4-zero-sums. Equivalently, its differential uniformity δ_S is 2.

3 Linearization

Definitions. We start by defining an approximation of an S-box. Throughout this work, we will only consider affine approximations.

Definition 4 (Intersection set, g -approximation). For $S, A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, we define the intersection set of S and A to be the set

$$S \cap A = \{x \in \mathbb{F}_2^n \mid S(x) = A(x)\}.$$

Furthermore, we say that A is a g -approximation of S if $|S \cap A| = g$.

The next definition is one of the key concepts in our study. We remark that previous works defined and studied vectorial *nonlinearity*, which most of the time takes values of the form $2^n - g$ for a small g (for functions with n -bit inputs). To simplify the expressions, we instead focus on the complementary notion of vectorial *linearity*, which is simply equal to g in the expression above.

Definition 5 (Vectorial linearity). For $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, we define its vectorial linearity VecLin_S as the maximum size of a subset $X \subseteq \mathbb{F}_2^n$ such that the restriction $S|_X$ can be extended to an affine map $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ (or, equivalently, as the maximum g such that there exists an affine g -approximation of S):

$$\begin{aligned} \text{VecLin}_S &= \max \{|X| \mid X \subseteq \mathbb{F}_2^n, S|_X = A|_X, A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \text{ affine}\} \\ &= \max \{|S \cap A| \mid A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \text{ affine}\} \\ &= 2^n - \min \{\text{wt}(S + A) \mid A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m \text{ affine}\}. \end{aligned}$$

The key problem we consider in this work is *linearization*, which consists in finding an affine g -approximation with g as large as possible.

Problem 1 (Linearization). Given $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, find an affine g -approximation of S with maximal g .

Bounds. The following trivial lower bound was given by Carlet.

Proposition 2 ([12, Prop. 6]). *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. Then, $\text{VecLin}_S \geq n + 1$.*

Proof. Choose an arbitrary affine basis $\mathcal{B}$ of $\mathbb{F}_2^n$ consisting of $n + 1$ points, on which an affine map is already fully determined by choosing an output value for each basis point. By choosing the output values according to the map S , we obtain an affine map A such that $S \cap A \supseteq \mathcal{B}$ and thus $|S \cap A| \geq n + 1$.

An interesting nontrivial bound based on the differential uniformity of the S-box was achieved by Nagy [33] and Ryabov [40] independently.

Proposition 3. *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ with differential uniformity δ_S . Then,*

$$\text{VecLin}_S \leq \sqrt{\delta_S} \cdot 2^{n/2} + \frac{1}{2}.$$

A similar (yet much weaker) bound exists for the Boolean linearity (see e.g. [28]).

Proposition 4. *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ with Boolean linearity Lin_S . Then,*

$$\text{VecLin}_S \leq 2^{n-1} + \frac{1}{2}\text{Lin}_S.$$

More specific bounds for APN and Bent functions are derived e.g. in [32].

Properties. Solutions to Problem 1 can easily be transferred to extended-affine-equivalent functions.

Proposition 5. *Let $S, S': \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ be such that $S' = B \circ S \circ A + C$ for some affine bijections $A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n, B: \mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ and an affine map $C: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$. Then, $\text{VecLin}_S = \text{VecLin}_{S'}$.*

Remark 1. Note that if S is bijective, then $\text{VecLin}_S = \text{VecLin}_{S^{-1}}$ is false in general. For example, for $n = 4$ and $S = [3, 7, 4, 8, 6, 11, 0, 5, 13, 10, 9, 1, 12, 2, 14, 15]$, we get $\text{VecLin}_S = 7$ and $\text{VecLin}_{S^{-1}} = 8$. This resolves [32, Problem 2].

The authors of [17] observed that for quadratic S-boxes, any approximation may be translated at the input by any constant, which follows from the fact that the derivatives of a quadratic function are affine. We derive a concrete expression since we will use it in specific optimizations. The proof is given in the full version.

Theorem 1 ([17, Thm. 3]). *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ have algebraic degree at most 2 and let A be an affine g -approximation of S with $X = S \cap A$. Then, for any $\Delta \in \mathbb{F}_2^n$, the map $A'(x) := A(x + \Delta) + \mathbf{D}_\Delta S(x)$ is also an affine g -approximation of S with $S \cap A' = X + \Delta$.*

3.1 Basic Algorithms

Basis Brute-Force Algorithm. A simple algorithm that theoretically solves Problem 1 is to interpolate possible affine approximations from the S-box values on each affine basis (unordered). The process is as follows: go through all (unordered) affine bases of $\mathbb{F}_2^n$; define in each step an affine map according to the output values of S on the current affine basis; check in how many points the defined affine map coincides with S . The number of unordered bases is given by $\mathcal{O}\left(\frac{2^{n(n+1)}}{(n+1)!}\right)$, which is cheaper than enumerating all $2^{m(n+1)}$ affine maps for $m \geq n$. It costs $\mathcal{O}(2^n)$ operations to count the number of points in the intersection of the interpolated affine map and the given S-box. This leads to the total complexity of $T_n = \mathcal{O}\left(\frac{2^{n(n+2)}}{(n+1)!}\right)$ operations on vectors. Clearly, for $n \geq 7$, this trivial algorithm becomes infeasible on a normal computer, as it requires more than $2^{45.9}$ operations ($2^{59.7}$ for $n = 8$).

Probabilistic Sampling Version. A more useful variant of the approach consists in *sampling* a random affine basis $B \subseteq \mathbb{F}_2^n$, interpolating the affine map A agreeing with S on B , and computing $S \cap A$. Despite its simplicity, it is a powerful method that is especially efficient in the presence of strong approximations. A pseudocode is provided in the full version of the paper. In practice, one may run the algorithm for a fixed number of iterations and keep the best found approximations.

Success Probability and Complexity. Let $g = \text{VecLin}_S$ be achieved by an affine approximation A with $X = S \cap A$. Then, the probability of sampling a basis candidate within X is given by (note that random $n + 1$ points form an affine basis with probability close to 29%, for simplicity we assume similar behavior within X)

$$\left(\frac{g}{2^n}\right)^{n+1} \approx \left(\frac{g}{2^n}\right)^{n+1} = (2^{n+1})^{\log_2 g - n}.$$

For an n -to- n S-box, a general affine map is defined by $(n + 1)n$ bits. Therefore, the sampling method yields the factor $1 - \log_2 g/n$ of complexity reduction (in bits) compared to direct brute-force. For example, for $n = 8$ and $g = 32$, the estimated complexity is equal to $2^{(n+1)(n-\log_2 g)} = 2^{27}$, while for $g = 16$ it increases to 2^{36} . For the case of vectorial linearity is close to $2^{n/2}$, we get the complexity estimate of $2^{(n+1)n/2}$ samples. Note however that, for large-output S-boxes, the vectorial linearity may be equal to $n + 1$, which results in sampling complexity close to quadratic in the exponent: $2^{(n+1)(n-\log_2(n+1))}$. The total time complexity has to be multiplied by the cost $\mathcal{O}(2^n)$ of testing a sample.

4 Extension Algorithms

4.1 Relations to Zero-Sums

Given $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, we know $\text{VecLin}_S \geq n + 1$ by Proposition 2. By carrying out the procedure described in the corresponding proof, be it only mentally or

actually in practice, we can hope to catch points outside our chosen affine basis for which A and S coincide as well. Let $\mathcal{B} = \{x_0, \dots, x_n\}$ be a chosen affine basis of $\mathbb{F}_2^n$ and $p \in \mathbb{F}_2^n \setminus \mathcal{B}$ such that $A(p) = S(p)$. Expressing p as an affine combination of the basis vectors x_i : $p = x_{i_1} + \dots + x_{i_j}$ with odd j , we obtain

$$S(p) = A(p) = A(x_{i_1} + \dots + x_{i_j}) = A(x_{i_1}) + \dots + A(x_{i_j}) = S(x_{i_1}) + \dots + S(x_{i_j}).$$

It follows that $\{p, x_{i_1}, \dots, x_{i_j}\}$ is a $(j+1)$ -zero-sum of S . If $j \in \{3, 5\}$, then, by Proposition 1, any j of the involved $j+1$ points are affinely independent and may constitute a part of the basis.

We can use this observation in the reverse direction: first find a k -zero-sum with $k \in \{4, 6\}$, choose any $(k-1)$ -element affine independent subset, and complete the basis arbitrarily. The equation above shows that the excluded k -th point of the zero-sum must satisfy the final affine approximation interpolated on the basis. The following proposition ensures the existence of at least one 4- or 6-zero-sum and thus of one extra point for the case $n \geq 4$ and $m \leq 2n-3$, based on a simple counting argument showing existence of 6-zero-sums for these parameters. The proof is deferred to the full version of the paper.

Proposition 6. *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, $n \geq 4$ and $m \leq 2n-3$. Then, $\text{VecLin}_S \geq n+2$.*

Since we are interested in collecting more than one extra point for the approximations, we will generalize this idea and encompass it into an iterative algorithm collecting extra points at each step. This replaces the ineffective arbitrary basis completion used above.

4.2 General Extension Algorithm

Following the discussion above, the idea is to gradually build up an affine basis $\mathcal{B} \subseteq \mathbb{F}_2^n$ of the domain, picking up as many “free” points as possible.

The idea is to start with a d -dimensional affine subspace $X_d \subset \mathbb{F}_2^n$ and an affine map $A_d: X_d \rightarrow \mathbb{F}_2^m$. For example, we can take as X_d the affine subspace generated by points (in $\mathbb{F}_2^n$) forming a 4-zero-sum ($d=2$) or a 6-zero-sum ($d=4$) of S , or simply a singleton ($d=0$).

There are $2^{n-d}-1$ affine subspaces parallel to X_d and we will merge X_d with one of them (name it $a + X_d$) to obtain a $(d+1)$ -dimensional affine subspace $X_{d+1} = X_d \sqcup (a + X_d)$. The next step is to extend the definition of the affine map $A_d: X_d \rightarrow \mathbb{F}_2^m$ to $A_{d+1}: X_{d+1} \rightarrow \mathbb{F}_2^m$ with $A_{d+1}|_{X_d} = A_d$. In order to gain a new point for the approximation, we need to have $A_{d+1}(x_d) = S(x_d)$ for some point in $X_{d+1} \setminus X_d$, i.e., in the chosen parallel affine subspace. It is easy to see that such x_d can be chosen arbitrarily. Furthermore, it completely defines the extension A_{d+1} since $A_{d+1}(x')$ for any $x' \in (a + X_d)$ can be deduced from

$$A_{d+1}(x_d) + A_{d+1}(x') = A_{d+1}(x_d - a) + A_{d+1}(x' - a) = A_d(x_d - a) + A_d(x' - a),$$

which is well-defined since both $x_d - a$ and $x' - a$ belong to X_d .

Observe that x_d by design satisfies the extended approximation with respect to S . What determines the other such points in $a + X_d$? The explicit condition is

$$S(x') \stackrel{?}{=} A_{d+1}(x') = A_d(x_d - a) + A_d(x' - a) + S(x_d).$$

The key observation is that it is symmetric in x' and $x' - a$, and, furthermore, induces a partition of $a + X_d$ into classes of points simultaneously fitting some extension A_{d+1} ! The proof is given in the full version of the paper.

Lemma 1. *Let $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, $d < n$ and X_d be a d -dimensional affine subspace $X_d \subseteq \mathbb{F}_2^n$, on which an affine map $A : X_d \rightarrow \mathbb{F}_2^m$ is defined. Let $a \in \mathbb{F}_2^n$ such that $X_d + a \neq X_d$. Then, the following is an equivalence relation on $a + X_d$:*

$$p \sim q \iff S(p) + S(q) = A(p - a) + A(q - a).$$

If we imagine the points of $\mathbb{F}_2^n \setminus X_d$ graphically and draw a line between two points p and q of the same coset exactly if $p \sim q$, we see that an equivalence class is nothing else than a *clique* (a full subgraph, meaning that every pair of points in the equivalence class is connected by a line). From now on, we will refer to it also as a clique. In this sense, each coset is partitioned into cliques. This means, according to Lemma 1, that for each clique c , there exists (in fact, a unique) affine map A_{d+1} which extends A_d onto the coset that contains c such that A_{d+1} and S coincide for all points of c , besides the points where A_d and S already coincide.

As mentioned before, we can choose an arbitrary coset $a + X_d$ and an arbitrary point inside it, meaning an arbitrary point x outside of X_d . Assigning $A_{d+1}(x) = S(x)$ extends affinely the definition of A_d . By the explicit condition above and its expression in the lemma, the full clique (i.e., the equivalence class of x) now belongs to the intersection $S \cap A_{d+1}$. It follows that, in particular, we may greedily choose the largest available clique at each step (this will be explored in Subsect. 4.3). Alternatively, we can recursively try a merge with every possible clique (this is explored in the full version of the paper).

Finally, we discuss how to update the cliques information after merging the cosets X_{d+1} and $a + X_{d+1}$ and extending the definition of A_d to A_{d+1} . Observe that, once we have $p \sim q$ in an extension step $X_d \subset X_{d+1}$ and $p, q \notin X_{d+1}$, we also have $p \sim q$ in the extension step $X_{d+1} \subset X_{d+2}$ because A_{d+1} is an extension of A_d . Thus, it is sufficient to check the edge condition according to Lemma 1 only for the new differences that arose during the extension $X_d \subset X_{d+1}$.

A practical and elegant way to achieve this is to precompute the differences $\Delta_x := p + q$ and $\Delta_y := S(p) + S(q)$ for each pair $(p, q) \in \mathbb{F}_2^n \times \mathbb{F}_2^n$ with $p \neq q$ and to create a lookup table by placing (p, q) at position (Δ_x, Δ_y) . This resulting table extends the usual DDT by storing the solution pairs of each corresponding differential equation, instead of their count. A small memory optimization consists in storing only pairs (p, q) with $p < q$.

Now, assume the table is precomputed, A_d is defined (for some $d > 0$) and we are merging X_d with $a + X_d$, so that A_{d+1} is defined on both of them. Let arbitrary $b \in X_d$ and define $L_d = \{x - b \mid x \in X_d\}$. The new equivalence edges

may only appear between affine subspaces $b + X_d$ and $b + a + X_d$ for some b , connecting a pair $(b, b + a + \Delta)$ of inputs for some $\Delta \in L_d$. Therefore, we are looking for values b such that

$$\Delta_y = A_{d+1}(b + \epsilon) + A_{d+1}(b + a + \Delta + \epsilon) = S(b) + S(b + a + \Delta).$$

This corresponds to the DDT entry with $\Delta_x = a + \Delta$ and Δ_y is derived from the expression above and A_{d+1} . Here, $\epsilon \in \mathbb{F}_2^n$ is an arbitrary vector required to reach the domain of A_{d+1} . By enumerating the DDT entry solutions for all 2^d values of such Δ_x , we obtain precisely all the edges $(b, b + \Delta_x)$ to be added to the relational graph.

4.3 Greedy Variant

We now present a greedy extension algorithm. At each of the $n + 1$ steps, we add another point to the approximation by choosing it greedily and randomly among points for which the size of the new approximation is maximized. The algorithm's pseudocode is given in Algorithm 1.

Implementation Details. We use the disjoint set union structure (DSU) to maintain and merge cliques. It has amortized cost close to constant for searches and merges. To keep track of the largest cliques, we maintain a min-heap data structure containing H largest available cliques for a small fixed parameter H , e.g. $H = 8$. The min-heap property allows us to efficiently replace the smallest clique with a bigger one. At the beginning of every iteration, we iterate through the full heap to find the largest clique outside of the main coset.

We maintain the sets $\text{LS}_x \subseteq \mathbb{F}_2^n$, $\text{LS}_{xy} \subseteq \mathbb{F}_2^n \times \mathbb{F}_2^m$ corresponding to the linear subspaces $X_d - X_d$ and $Y_d - Y_d$, where $Y_d = \{(x, A_d(x)) \mid x \in X_d\}$. The former one is used for testing if two inputs are in the same coset; the latter one is used to enumerate the graph of A_d , which allows inferring all possible pairs (Δ_x, Δ_y) in the discussion from the previous subsection.

For starting with a given 4- or 6-zero-sum, one can force the initial choice of a and the choices of a' in the first $2/3$ iterations.

Complexity Analysis. The loop $v \in \text{LS}_{xy}$ dominates the complexity of the algorithm. It iterates over 2^d elements in iteration d . For each element v , the algorithm goes through all solutions to the corresponding difference equation, which are located in DDT_{sol} at position $(\tilde{\Delta}_x, \tilde{\Delta}_y)$. We use a simplifying assumption that the involved differences $\tilde{\Delta}_x, \tilde{\Delta}_y$ are random (the one corresponding to $v = 0$ is special due to the fact that it connects the largest cliques, while the others are just shifts of it by a subspace). Then, on average we have 1 solution, for which we query the DSU structure (we assume the constant cost although in theory it is shown to be the inverse Ackermann function, which is less than 5 for any representable number), check an LS_x membership via a lookup and update the small heap in time $\mathcal{O}(\log H)$. We conclude with the complexity $\mathcal{O}(\log H \cdot (2^0 + 2^1 + \dots + 2^{n-1})) = \mathcal{O}(\log H 2^n) = \mathcal{O}(2^n)$ since H is a small

Algorithm 1 Extension algorithm (greedy)**Inputs:** function $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, parameter $H \in \mathbb{Z}_{>0}$ **Output:** affine approximation $A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ of S

```

1:  $\text{DDT}_{\text{sol}} \leftarrow \{(\Delta_x, \Delta_y) \mapsto \{x \mid S(x) + S(x + \Delta_x) = \Delta_y\}\}$   $\triangleright$  DDT with solutions
2:  $\text{LS}_x = \{0\} \subseteq \mathbb{F}_2^n$   $\triangleright$  Linear space of guessed input differences
3:  $\text{LS}_{xy} = \{(0, 0)\} \subseteq \mathbb{F}_2^n \times \mathbb{F}_2^m$   $\triangleright$  Linear space of guessed input and output differences
4:  $a \xleftarrow{\$} \mathbb{F}_2^n$   $\triangleright$  Main clique representative
5:  $\text{cliques} \leftarrow \{\{x\} : x \in \mathbb{F}_2^n\}$   $\triangleright$  DSU data structure
6:  $\text{top} \leftarrow$  min-heap initialized with  $H$  random elements of  $\{1\} \times \mathbb{F}_2^n \setminus \{a\}$ 
7: for  $d \in \{0, \dots, n-1\}$  do
8:    $a' \leftarrow$  largest element of  $\text{top}$  (via iteration)
9:    $(\Delta_x, \Delta_y) \leftarrow (a + a', S(a) + S(a'))$ 
10:   $\text{LS}_x \leftarrow \text{LS}_x \cup (\Delta_x + \text{LS}_x)$ 
11:  Remove pairs  $(s, x)$  from  $\text{top}$  with  $(a + x) \in \text{LS}_x$ 
12:  for all  $v \in \text{LS}_{xy}$  do
13:     $(\tilde{\Delta}_x, \tilde{\Delta}_y) \leftarrow v + (\Delta_x, \Delta_y)$ 
14:    for all  $x \in \text{DDT}_{\text{sol}}[\tilde{\Delta}_x, \tilde{\Delta}_y]$  do
15:      Try to merge cliques containing  $x$  and  $x + \tilde{\Delta}_x$  in cliques
16:      if merge occurred and  $(a + x) \notin \text{LS}_x$  then
17:        Insert the pair (size, representative  $x$ ) of the new clique into  $\text{top}$ 
18:        If  $\text{top}$  has size  $H + 1$ , remove the smallest element from it
19:      end if
20:    end for
21:  end for
22:   $\text{LS}_{xy} \leftarrow \text{LS}_{xy} \cup ((\Delta_x, \Delta_y) + \text{LS}_{xy})$ 
23: end for
24:  $A \leftarrow$  affine map interpolated from the largest clique (or from its graph  $\text{LS}_{xy}$ )
25: return  $A$ 

```

constant. This is complexity per execution of the algorithm (that is, per one sampled affine approximation). This is a very high performance since just reading an S-box would take this amount of computations (although, we require random accesses to the precomputed DDT). Its greedy heuristic makes it produce much stronger results than pure sampling (Subsect. 3.1) in practice, at essentially the same cost.

5 Reduction Algorithms

In this section, we present algorithms based on iterative *reduction* of the full input set down to an affine approximation set. These algorithms also have a spectral component, namely, they employ the analysis of the Walsh transform of the target function. We describe two variants of the algorithm: an exhaustive reduction and a greedy reduction.

The starting idea for this direction is that the vectorial linearity can be seen as a stronger version of the usual (Boolean) linearity: all the output bits of the function must be approximated simultaneously on the same set of inputs. In

particular, a strong vectorial approximation should manifest itself in Boolean approximations. Indeed, if $S(x) = A(x)$ for g different inputs x , then (in particular) the first output bit of S can be approximated by $A_1(x)$ with at least g matching inputs. This creates a bias towards the approximation $S_1 = A_1$, which may become the best approximation for this bit, unless the remaining $2^n - g$ inputs are specifically canceling the bias, or another better approximation exists for the bit. Although this observation does not provide guarantees, it can still be used as a heuristic to guide the search.

5.1 Exhaustive Variant

In this variant, we will recursively enumerate candidates for the affine maps $A_1, A_2, \dots, A_m: \mathbb{F}_2^n \rightarrow \mathbb{F}_2$ approximating the coordinates $S_1, \dots, S_m$ of S one by one. Each such candidate A_i is defined by a *mask* $\alpha_i \in \mathbb{F}_2^n$ and a constant $c \in \mathbb{F}_2$ such that $A_i(x) = \langle \alpha_i, x \rangle + c$. During the enumeration, we will maintain the subset $X \subseteq \mathbb{F}_2^n$ of inputs satisfying the current guesses of $A_1, \dots, A_d$ at the recursion depth d . The subsets X at the final depth m correspond to the desired intersections $S \cap A$. The algorithm together with optimizations described below is presented in Algorithm 2.

Algorithm 2 Exhaustive reduction algorithm

Inputs: function $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, lower bound $\text{LB} \in \mathbb{Z}_+$, set of affine derivatives $\{\mathbf{D}_{\Delta_i} S\}_i$

Output: affine approximation(s) $A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, with at least LB matching points

```

1: function EXHAUSTIVEREDUCTION( $d, L, X$ )
2:   if  $|X| < \text{LB}$  then
3:     return ▷ Cut search branch
4:   end if
5:   if  $d = m$  then
6:     Report an affine approximation  $A$  with  $S \cap A = X$ 
7:     return
8:   end if
9:   for all  $\alpha \in \mathbb{F}_2^n$  do
10:     $L' \leftarrow (L \parallel \langle \alpha, x \rangle): \mathbb{F}_2^n \rightarrow \mathbb{F}_2^{d+1}$ 
11:    if  $L'$  is lex. smallest among  $L' + \{\mathbf{D}_{\Delta_i} S\}_i$  then ▷ as matrices,
12:      ▷ ignoring the constant part
13:       $X'_0 \leftarrow \{x \in X \mid \langle \alpha, x \rangle + S_{d+1}(x) = 0\}$ 
14:       $X'_1 \leftarrow \{x \in X \mid \langle \alpha, x \rangle + S_{d+1}(x) = 1\}$ 
15:      EXHAUSTIVEREDUCTION( $d + 1, L', X'_0$ )
16:      EXHAUSTIVEREDUCTION( $d + 1, L', X'_1$ )
17:    end if
18:  end for
19:
20: end function
21: EXHAUSTIVEREDUCTION( $0, (), \mathbb{F}_2^n$ )
    
```

Upper-Bounding the Vectorial Linearity. The first important optimization that we add to the process is cutting off search subtrees for which the subset X of satisfying inputs is smaller than the lower bound parameter LB given as input. This trick relies on the fact that the subset X only reduces with the recursion depth. As we shall see, for the reasonable bound $\text{LB} = 2^{n/2}$, this amounts to roughly halving the search space in bits. If the algorithm finishes without finding any approximation with at least LB points, it follows that the vectorial linearity of the S-box is at most $\text{LB} - 1$. That is why we say that the algorithm has the *upper-bounding property*, while it receives the *lower bound* as input.

In practice, one would first run fast heuristic sampling algorithms (sampling, greedy extension or greedy reduction) and record the maximal size g of observed approximations. Then, the exhaustive algorithm is run with the lower bound $\text{LB} = g + 1$ in order to verify that the vectorial linearity of the S-box is either exactly g (if the exhaustive algorithm does not find an approximation) or higher (if the exhaustive algorithm finds an approximation). In case the latter execution is not feasible, the LB parameter may be increased so that the exhaustive algorithm at least yields an upper bound on the vectorial linearity.

Optimization for Quadratic S-Boxes. Using Theorem 1, we can optimize the search by a factor up to 2^n . Recall that, if A is a g -approximation of S , then $A'(x) = A(x + \Delta) + \mathbf{D}_\Delta S(x)$ is also a g -approximation, for any nonzero Δ . This means that, up to a constant independent of x (note that $A(x + \Delta) = A(x) + A(\Delta) + A(0)$), it is sufficient to check one representative of all translations

$$\{A(x) + \mathbf{D}_\Delta S(x) \mid \Delta \in \mathbb{F}_2^n\},$$

together with all possible constant additions. We consider only the linear parts of the derivatives (which correspond to quadratic parts of the original function). If the linear space of all (linear parts of) derivatives has full dimension n , we can check one out of 2^n candidates for the linear part from the above set. However, in order to use this property during the exhaustive reduction algorithm, we need to recognize the representatives before going too deep into the recursion. For simplicity, we will assume that the S-box has a small number (typically, 1-3) of components which yield full dimension n of the linear parts of the derivatives. Then, we guess these components first in the recursion, and check if the current guess A' is lexicographically the smallest among the equivalent guesses $\{A' + \mathbf{D}_\Delta S\}_{\Delta \in \mathbb{F}_2^n}$ (only considering the linear part of the map), for depth $d < 3$. This allows to cut off the equivalent search subtrees early and yields a factor 2^n reduction to the overall search cost.

Unless the S-box has an extremely sparse set of quadratic monomials, we expect our assumption to hold (and it was in practice in all our experiments). For example, if the matrix describing the quadratic monomials of a coordinate has full rank n , it is sufficient to check this coordinate first. With two coordinates, this is even more likely to be true.

Complexity Analysis. Let d be the current recursion depth, i.e., we have guessed 2^d masks and constants ($n + 1$ bits per level). If we only fix d masks, the 2^d

assignments to the constants partition the input space $\mathbb{F}_2^n$. Therefore, each set X at the current level has an average size 2^{n-d} . As a result, we estimate that most of the search subtrees will be cut shortly after depth d satisfying $2^{n-d} < \text{LB}$, equivalent to $d > n - \log_2 \text{LB}$.

This leads to the complexity estimation $2^{(n+1)(\lceil \log_2 \text{LB} \rceil + \varepsilon)}$ for a small integer ε . Focusing on the case when **VecLin** is close to $2^{n/2}$, we get the complexity $2^{(n+1)(n/2+\varepsilon)}$. Note that for even n , setting $\varepsilon = 0$ does not lead to sufficient filtering as the depth is right on the boundary, and therefore, we need $\varepsilon \geq 1$ to ensure strong filtering; for odd n , this is not necessary. We conclude with the complexity estimate (denoted T_{nodes}) $2^{(n+1)\lfloor n/2+1 \rfloor}$ recursion calls. This complexity is similar to that of the sampling algorithm; however, the new algorithm is guaranteed to exhaust the full search space and thus can be used to compute **VecLin_S** precisely or at least get an upper bound. Note that the complexity in terms of operations requires an extra factor of $2^{1.5n}$ for enumerating 2^{n+1} candidates masks and filtering the set of inputs of size about $2^{0.5n}$ at depth $d = n/2$, yielding total estimate of $\mathcal{O}(2^{(n+1)\lfloor n/2+2.5 \rfloor})$ operations.

For a more precise analysis, we model the partitioning into 2^d sets as fully random (which is plausible since most of the work is spent on random linear masks dividing the set X randomly). We also analyze each single bucket of the partition independently of others. For each input from the domain $\mathbb{F}_2^n$, the probability for it to land in the chosen bucket at depth d is equal to 2^{-d} . Modeling all the inputs independently, we use the binomial distribution to compute the distribution of the number of inputs in the chosen bucket, denoted $p_{d,i}$:

$$p_{d,i} = \Pr[|X| = i \text{ after depth} = d] = \binom{2^n}{i} (2^{-d})^i (1 - 2^{-d})^{2^n - i}.$$

The next 2^{n+1} candidates in each recursive call are computed explicitly before filtering them by **LB**, adding an extra 2^{n+1} factor compared to the number of candidates after filtering. This gives an accumulated complexity T_d at depth d :

$$\begin{aligned} T_d &= 2^{(n+1)(d+1)} \cdot \Pr[|X| \geq \text{LB} \text{ after depth} = d] \\ &= 2^{(n+1)(d+1)} \cdot \sum_{i=\text{LB}}^{2^n} \binom{2^n}{i} (2^{-d})^i (1 - 2^{-d})^{2^n - i}. \end{aligned}$$

The total complexity is then estimated as the sum of accumulated per-round complexity. For simplicity, we measure it in the number of visited nodes (denoted T_{nodes}). This ignores the different average X sets sizes at different depths, which we set to $2^{n/2}$ to get an approximation for the complexity in terms of word operations (denoted $T = T_{\text{nodes}} 2^{n/2}$). We conclude with the expression

$$T_{\text{nodes}} = \sum_{d=1}^n 2^{(n+1)(d+1)} \cdot \sum_{i=\text{LB}}^{2^n} \binom{2^n}{i} (2^{-d})^i (1 - 2^{-d})^{2^n - i}.$$

While the earlier estimate $2^{(n+1)\lfloor n/2+1 \rfloor}$ can be quickly used to estimate the feasibility of the search, this more precise expression needs to be computed on a

computer (note that it is independent of the S-box). We found that it matches quite well the experimental results.

5.2 Greedy Variant

By analogy with the greedy extension algorithm, we can choose output coordinates of A that locally (at the current step) maximize the size of the resulting set X' . Furthermore, we can choose to guess *components* (i.e., linear combinations of coordinates) since this may lead to larger sets X' .

For the purpose of optimizing the resulting implementation, we will present an LAT-based formulation of the method, where we will no longer track the intermediate sets X but rather track their size using LAT coefficients. This representation is based on an observation from [28], rewritten in our terminology.

Theorem 2 ([28, Proof of Thm. 4.3]). *Let $S, A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, where $A = L + c$, L is linear and $c \in \mathbb{F}_2^m$. Then,*

$$\sum_{\beta \in \mathbb{F}_2^m} (-1)^{\langle \beta, c \rangle} W_S(\langle \beta, L \rangle, \beta) = \sum_{\beta \in \mathbb{F}_2^m} \sum_{x \in \mathbb{F}_2^n} (-1)^{\langle \beta, S(x) + A(x) \rangle} = 2^m |S \cap A|.$$

The theorem says that we can compute the strength of an approximation A (i.e., the size of $S \cap A$) simply by summing up LAT coefficients, where in each column β , we take one coefficient corresponding to the coordinate of the linear part of the affine map (i.e., $\langle \beta, L \rangle$), negated if the inner product of β and c is equal to 1. We can also consider an m' -dimensional linear subspace $V \subseteq \mathbb{F}_2^m$ in the output, where $m' < m$. Then the corresponding sum will yield the size of the set $X = (\pi \circ S) \cap (\pi \circ A)$, where π is the projection of $\mathbb{F}_2^m$ on V discarding the orthogonal complement $V^\perp$, i.e., a partial output linearization.

The pseudocode for the algorithm is given in Algorithm 3. We start by precomputing the full LAT of the S-box, denoted C_0 . During the algorithm, we will “compress” it by summing along m independent directions corresponding to output masks. The process consists of m consequent steps. Intermediate tables corresponding to LAT sums over guessed subspaces are denoted by C_d at depth d . These tables have signature $\mathbb{F}_2^n \times \mathbb{F}_2^{m-d} \rightarrow \mathbb{Z} : (\alpha, \beta) \mapsto w$, where β is a representative of a coset of the currently guessed d -dimensional linear subspace.

In each step, we choose a pair $(\tilde{\alpha}, \tilde{\beta})$, $\tilde{\beta} \neq 0$, with the highest absolute value of $w = C_d[\tilde{\alpha}, \tilde{\beta}]$, where the ties are resolved at random. Each such choice corresponds to the guess $\langle \tilde{\beta}, A(x) \rangle = \langle \tilde{\alpha}, x \rangle + [w < 0]$ of one component of the desired affine approximation A . Here, $[w < 0]$ is the Kronecker delta, equal to 1 if w is negative, and 0 otherwise. Then, we compress the table C_d into C_{d+1} by setting

$$C_{d+1}[\alpha, \beta] = C_d[\alpha, \beta] + C_d[\alpha + \tilde{\alpha}, \beta + \tilde{\beta}]$$

for all $\beta < \beta + \tilde{\beta}$ (according to an arbitrary predetermined order).

The constraint $\beta \neq 0$ ensures that all the chosen β values are linearly independent and the affine approximation A is well-defined at the end of the algorithm. Furthermore, from Theorem 2, we can deduce that $C_m[0, 0]$ (which stores the sum of LAT over the guessed subspace) is equal to $2^m |S \cap A|$.

Algorithm 3 Greedy reduction algorithm

Inputs: function $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$, parameters $t_d, t_t \in \mathbb{Z}_{>0}$
Output: affine approximation $A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$

```

1:  $C_0 \leftarrow \{(\alpha, \beta) \mapsto W_S(\alpha, \beta)\}$  ▷ LAT can be precomputed
2:  $A \leftarrow \{\}$ 
3: for all  $d \in \{0, \dots, m-1\}$  do
4:   if  $d < t_d$  then
5:      $t \leftarrow t_t$ 
6:   else
7:      $t \leftarrow 1$ 
8:   end if
9:    $(\tilde{\alpha}, \tilde{\beta}, w) \leftarrow$  randomly chosen pair from the largest  $t$  coefficients (by the absolute
      value) in  $C_d$  and the associated sum  $w$ ,  $\tilde{\beta} \neq 0$ .
10:   $A \leftarrow A \cup \{(\tilde{\alpha}, \tilde{\beta}, w)\}$  ▷  $\langle \tilde{\beta}, A(x) \rangle = \langle \tilde{\alpha}, x \rangle + [w < 0]$ 
11:   $C_{d+1} \leftarrow \{(\alpha, \beta) \mapsto C_d[\alpha, \beta] + \text{sgn}(w) \cdot C_d[\alpha + \tilde{\alpha}, \beta + \tilde{\beta}] \mid \beta < \beta + \tilde{\beta}\}$  ▷ merge
12: end for ▷ Note that  $C_m[0, 0] = 2^m |S \cap \tilde{A}|$ 
13: return affine approximation  $\tilde{A}$  reconstructed from  $A$ 
    
```

Increasing Variability. As we will show in the experiments, the fully-greedy algorithm often yields low variability in its answers, since often the involved maximal absolute biases are given by a small number of masks, and the algorithm gets stuck in the same approximations being local maximums in the search space. In order to tackle this problem, we explore the first t_d masks from top- t_t masks by the absolute correlation *equiprobably*. For example, the parameters $(t_d, t_t) = (4, 5)$ make the algorithm consider top-5 masks equally for the first 4 guesses and top-1 masks for the remaining guesses.

Complexity Analysis. Although the LAT can be precomputed once, the complexity of the algorithm is dominated by the first compression step, which maps the table C_0 of size 2^{n+m} to a table half of that size. Thus, we get $2^{n+m-1-d}$ additions at iteration d which sums to $\mathcal{O}(2^{n+m})$ complexity of one execution. Note that maintaining the largest t_t coefficients and the corresponding mask pairs can be done efficiently using the heap data structure, adding a constant overhead per each computed element of the table C_{d+1} .

6 Experimental Evaluation

In this section, we report results of the application of our algorithms to a wide range of S-boxes from the literature.

Procedure. The typical procedure consists of running the greedy extension algorithm to get some first heuristic results, initially starting with random 4-zero-sums (or 6-zero-sums for APNs), and then with random single points. The main reasons why we preferred the extension algorithm over the reduction algorithm

are the efficient sampling complexity and inherent variability of the greedy extension algorithm; it also yielded better approximations in most cases. Sometimes we ran the greedy reduction algorithm to have better coverage (the only cases when it improved the approximation was the case of the inverse map on even bits). The two algorithms were initially implemented in SageMath [41], later with a port of the greedy extension algorithm to Rust with Python bindings.

After obtaining a heuristic approximation, if feasible (for most cases of $n \leq 8$), we ran the exhaustive reduction algorithm to investigate whether the initial values resulted from the greedy algorithms indeed already gave us the vectorial linearity. The algorithm was implemented in C++ due to its simplicity and performance (parallelization) requirements.

Experimental Setup. All heuristic algorithms for S-boxes were run on a single core of a modern laptop, typically from a few minutes to a few hours per S-box. Exhaustive algorithms were executed on a high-performance cluster (a few nodes of 128-core AMD EPYC 7452 processors) in order to process a large number of instances in a short time. Since the algorithm does not require a lot of memory, it can in principle be run on normal PC/laptop as well (however, for 16-bit Super-Sboxes, the precomputed DDT requires about 40 GB of RAM).

6.1 Monomial S-Boxes

The seminal work of Nyberg [35] showed that monomial maps over the finite field $\mathbb{F}_{2^n}$ can be used as strong S-boxes. In particular, the cube mapping $x \mapsto x^3$ is always APN although it is only a bijection for odd n . The multiplicative inverse mapping $x \mapsto x^{2^n-2}$ is differentially 4-uniform for even n and APN for odd n , and always a bijection, which made it a popular choice in symmetric-key designs, including the standard AES block cipher (up to affine equivalence). The algebraic degrees of these maps are 2 and $n-1$, respectively. We also consider the quadratic S-box given by $x \mapsto x^5$, which is differentially 4-uniform for even n and APN for odd n as well, but may have a cheaper implementation cost. In addition, we consider the monomial $x^{2^{n/2-1}+1}$ for which Nagy [32] showed an approximation of size $2^{n/2} + 2$ and an upper bound $\sqrt{2} \cdot 2^{n/2} + \frac{1}{2}$. The results of applying our procedure to these S-boxes for various values of n is given in Table 2. All of them were obtained within 2 days of runtime per S-box (time to the result gradually increasing with n ; for $n \leq 8$ a few minutes are sufficient).

For the x^9 map, it is surprising that for $n = 6$ the vectorial linearity is equal to 15 while for $n = 7$ it is lower (equal to 11). Note that these are exact values since we were able to verify them using exhaustive algorithms.

For $n = 10$, the smallest (among maximum) approximation we found was 22 points again for the x^9 map (note that this is significantly smaller than the general disproven conjectured bound of $2^{n/2} + 2 = 34$, yet we cannot claim that the value is exact). Furthermore, for $n = 9$ we achieved the same value of 22 points. From $n = 12$ to $n = 13$, the size of the approximation for x^9 decreased again according to our results.

Table 2. Vectorial linearity of monomial functions x^e , $e \in \{3, 5, 7, 9, 2^n - 2\}$. Values for $n \leq 8$ are exact, values for $n \geq 9$ are lower bounds.

S	n															
	4	5	6	7	8	9	10	11	12	13	14	15	16			
x^3	6	7	9	11	16	19	25	28	36	40	55	51	93			
x^5	7	7	10	12	16	18	28	26	46	35	55	44	67			
x^7	7	7	13	16	22	28	34	32	46	41	58	55	76			
x^9	6	7	15	11	18	22	22	27	43	36	42	78	47			
$x^{2^{n/2-1}+1}$	6		10		18		34		66		130		258			
x^{2^n-2}	7	7	10	12	18	19	34	30	66	42	130	53	91			

All these surprising heuristic findings raise the question of further theoretical or computational investigation.

Linearization of the Inverse Mapping for Even n . The authors of [44] described an elegant generic way to linearize the multiplicative inverse mapping for even values of n , based on the decomposition $x^{2^n-2} = (x^{2^{n/2}+1})^{2^{n/2}-2} \cdot x^{2^{n/2}}$. Later, [32] also described a similar approximation of the same size for the monomial $x^{2^{n/2-1}+1}$, which is APN for $n = 4k$. It follows that for even n , $\text{VecLin}_{x^{-1}} \geq 2^{n/2} + 2$. These are precisely the values that we observed using our algorithms for $6 \leq n \leq 14$. For $n = 16$ we were unable to recover the known approximations of size 258 within 2 days of computation, although we expect that it should be possible with more effort. In fact, for smaller dimensions n , the inverse monomial was always the slowest for our greedy extension algorithm to converge to a result. Furthermore, the inverse monomial is one of only a few cases where the greedy reduction algorithm found the best approximation faster. However, we do not have an optimized implementation of it that could be ran on $n = 16$.

Furthermore, for $n = 8$, we were able to verify that 18 points is indeed the best possible linearization, using the exhaustive reduction algorithm (it recorded visiting 2^{44} nodes).

This leaves open the question of how the best approximation of the inverse mapping is structured for odd n . In the full version, we prove that, in this case, the decomposition cannot be used to obtain a linearization.

6.2 S-Boxes from Ciphers

We also applied our algorithms to S-boxes from the literature, namely from cryptographic designs of block/stream ciphers [38]. A selection of results is presented in Table 3.

We observe that for 8 bits, the strongest permutation is the AES S-box, based on the inversion in the finite field. However, there are quite a few other S-boxes which are built using totally different methodologies and have a close value of vectorial linearity 19–22. For example, Kalyna S-boxes [25] were generated using

Table 3. Vectorial linearity of S-boxes from the literature.

n	S-box	δ_S	Lin_S	$\deg S$	VecLin_S
5	Fides ₅	2	8	2	7
	Ascon	8	16	2	11
6	Fides ₆	2	16	4	9
	SC2000 ₆	4	16	5	10
7	WAGE	8	40	6	17
8	AES	4	32	7	18
	Kalyna π_2, π_3	8	48	7	19
	Kalyna π_0, π_1	8	48	7	20
	BelT	8	52	7	20
8	Kuznyechik	8	56	7	20
	Anubis	8	68	7	21
	Skipjack	12	56	7	21
	Scream	8	64	6	22
	SNOW-3G SQ	8	64	5	26
	iScream	16	64	6	27
9	Safer	128	92	7	30
	Skinny ₈	64	128	6	47
	CSS	128	256	4	81
	DryGASCON ₂₅₆	130	320	6	77

hill-climbing/gradient-descent; BelT S-box is an exponentiation in the finite field and Kuznyechik’s S-box has a related but different structure [37]. Skipjack’s design is secret but it was suggested that it was also optimized for a flatter Walsh spectrum distribution [8]. We conclude that there are multiple approaches to obtain S-boxes with reasonable vectorial linearity.

We now discuss a few S-boxes with high vectorial linearity. The CSS (Content Scrambling System) S-box used in the old DVD protection mechanisms has an extremely sparse ANF and multiple linear components, thus it is unsurprising that it can be linearized with probability almost 1/3. The SKINNY 8-bit S-box was optimized to be lightweight and can be implemented using 8 NOR and 8 XOR gates, which made it quite weak overall, leading to strong linearization too (probability about 18% for the full S-box).

6.3 Super-Sboxes from Ciphers

We recall that a Super-Sbox consists of a layer of four S-boxes, key addition, a MixColumn linear layer, and another layer of four S-boxes, having 8 S-boxes in total. By an abuse of terminology, we also include the MISTY-FI function which is an unbalanced Feistel Network with 3 S-boxes and a 16-bit key, possessing similar properties as Super-Sboxes. We ran our greedy extension algorithm on several 16-bit Super-Sboxes and the MISTY-FI function, with zero and random keys. The results are given in Table 4. In addition, to test the limits of our methods, we ran a simplified variant of the Greedy-Reduction algorithm on the 32-bit Super-Sbox of AES, which attracted a lot of interest in analysis.

For Midori64, it is noticeable that the less distinct the four individual values are, the higher the vector linearity. In the case of complete equality, we get $\text{VecLin} \geq 4096 = 2^{12} = 8^4$. Surprisingly, this is greater than just four 4-bit Midori S-boxes run in succession yielding $\text{VecLin} = 7^4$, since one such individual

Table 4. Lower bounds on vectorial linearity for the 16-bit Super-Sboxes of Midori, SKINNY, LED, the MISTY-FI keyed function, and the 32-bit Super-Sbox of AES.

Super-Sbox Key			Super-Sbox Key		
		VecLin $\geq$			VecLin $\geq$
Midori64	(0,0,0,0)	4096	LED	(0,0,0,0)	122
Midori64	(C,C,C,C)	4096	LED	(5,A,5,9)	123
Midori64	(E,2,1,9)	576	MISTY-FI	(0,0,0,0)	50
Midori64	(0,A,0,F)	1280	MISTY-FI	(0,8,0,e)	50
SKINNY64	(0,0,0,0)	1060	AES	(00,00,00,00)	60
SKINNY64	(9,8,2,0)	1046			

4-bit S-box has a vectorial linearity of 7. After investigation, we found a simple reason for this: the approximations come from the involutive property of the S-box which occurs when the four key nibbles are the same. In this case, the approximation is simply the identity map. On the other hand, for a random key, such as (E,2,1,9), the best approximation we found is not based on fixed points since the Super-Sbox does not have any.

For SKINNY64, we found strong approximations independently of the key used, although the approximations themselves do depend on it. Our investigation revealed two weaknesses that led to the distinguisher. First, the sparse linear layer (containing only three 4-bit nibble XORs and a nibble permutation) implies one composition of two S-boxes that has no modification in-between (except the key/constant XOR). Furthermore, fixing one input nibble to a constant creates two more such fixed compositions. Second, compositions of the S-box with itself have VecLin ranging from 7 to 13 (depending on the constant in-between), while the S-box itself has VecLin of 7. In the approximation we found, about half of the inputs have the third nibble fixed, which leads to another S-box composition with VecLin equal to 13.

Interestingly, the MISTY-FI function leads with the smallest approximations we could find. An explanation is that it is built from three monomial S-boxes over two different fields ($\mathbb{F}_{2^7}$ and $\mathbb{F}_{2^9}$). The LED Super-Sbox is also strong, which is explained by its strong MDS linear layer and the optimal PRESENT S-box.

For the 32-bit AES Super-Sbox, we managed to find an approximation of size 60. While it is not strong compared to the domain size, it is a nontrivial result and we are not aware of other techniques that could achieve this. Clearly, we cannot make conclusions on the vectorial linearity of the Super-Sbox in this case, and it remains a challenging open question. We invite other researchers to improve our result.

6.4 APNs

We recall two equivalence notions. Two vectorial Boolean functions $F, G : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ are said to be affine equivalent if there exist affine bijections $A : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n, B :$

$\mathbb{F}_2^m \rightarrow \mathbb{F}_2^m$ and an affine map $C : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$ such that $G = (B \circ F \circ A) + C$. Furthermore, F, G are said to be CCZ-equivalent [14] if there is an affine bijection $M : \mathbb{F}_2^{n+m} \rightarrow \mathbb{F}_2^{n+m}$ such that

$$M \times \{(x, F(x)) | x \in \mathbb{F}_2^n\} = \{(x, G(x)) | x \in \mathbb{F}_2^n\}.$$

We applied our algorithms to several known APN extended affine (EA) equivalence classes. Brinkmann and Leander [11] classified all APN functions up to extended-affine equivalence and CCZ-equivalence for n up to 5 bits. Although they showed that all APN functions in dimensions up to 5 are CCZ-equivalent to a monomial mapping (which we covered in Subsect. 6.1), vectorial linearity is not necessarily preserved by CCZ-equivalence, which is why we checked the EA-equivalence classes. However, when $3 \leq n \leq 5$, we can use our lower bound $n + 2$ for $n \geq 4$ (and $n + 1$ for $n = 3$), which coincides with the Sidon-set upper bound applicable to APN functions (see [32]). Thus, the vectorial linearity of APN functions is forced to be equal to 4, 6, 7 respectively for $n = 3, 4, 5$ simply by the theoretical bounds. We also verified this using our algorithms.

In the case of $n = 6$, we considered the 534 EA-classes of cubic APNs given by Langevin [27]. With the exhaustive reduction algorithm, we confirmed that all of them have a vectorial linearity equal to 9 (the same as the 6-bit APN permutation).

For $n = 7$, we checked the APNs provided in the `sboxU` package. Most of the functions had vectorial linearity 11 (same as the cube mapping), but some of them reached the maximum Sidon set size bound of 12.

For $8 \leq n \leq 10$, we checked the recently found quadratic APNs from [6]. Interestingly, we found a few 8-bit APN functions with vectorial linearity 15, which is smaller than any of the 8-bit monomial mappings. An example of such a function, denoted S_{15} , is given in the full version of the paper. It has Boolean linearity 32.

6.5 Discussion

The authors of [28] conjectured a general bound for arbitrary maps $S : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^m$:

$$\text{VecLin}_S \geq 2^n - (1 - 2^{-m}) (2^n - 2^{n/2}).$$

For $m = n$, the right-hand side becomes $2^{n/2} + 1 - 2^{-n/2}$, which means $\text{VecLin}_S \geq 2^{n/2} + 1$ for even n . For example, for $m = n = 8$, this is equal to 17. Very recently, Nagy [32] disproved the conjecture by using relations between APN functions and Sidon sets and some computations, showing that $\text{VecLin}_{x^3} \leq 16$ for $n = 8$ and $\text{VecLin}_{x^3} \leq 21$ for $n = 9$. However, Nagy was not able to find matching approximations.

Our results confirm that $S(x) = x^3$ has $\text{VecLin}_S = 16$ precisely (with independent upper bound verification), and that the function $S = S_{15}$ has $\text{VecLin}_S = 15$ precisely, further departing from the conjecture by one. We recall that the proven lower bound for $n = 8$ is 10. It is thus a challenging open problem to find 8-bit S-boxes with vectorial linearity lower than 14, or to prove that they do not exist.

7 Applications to Cryptanalysis

Linearization of S-boxes is a useful tool that has strong potential to aid other cryptanalysis techniques and key recovery processes. The current state-of-the-art has particularly highlighted its relevance in the setting of low-data attacks, especially when only one plaintext-cipher pair is available, or in the similar CICO problem for permutations [22].

Problem 2 (CICO). Let $F: (\mathbb{F}_2^n)^m \rightarrow (\mathbb{F}_2^n)^m$ be a bijection and let c, r be positive integers with $c+r = m$. The $\text{CICO}_{c,r}$ problem for F asks to find an input-output pair $y = F(x)$ such that the first c $\mathbb{F}_2^n$ -entries of x and y are equal to zero, that is, $y_0 = \dots = y_{c-1} = x_0 = \dots = x_{c-1} = 0 \in \mathbb{F}_2^n$.

7.1 Covering Domain with Approximations

In the single-plaintext setting, aiming for all S-boxes to be successfully linearized with a single best approximation is not the best strategy, since it will quickly lead to a low success probability after trying to linearize a few S-boxes in the attacked primitive. The idea is to perform a new attack attempt without requesting new encryptions but with guessing a different set of approximations for S-boxes. In this way, we will ensure that for any existing solution to the problem, we have a guaranteed success probability of our attack which only depends on the randomness sampled during the attack.

We consider two solutions to the problem: a deterministic one and a probabilistic one. For both approaches, we will assume that a relatively large number of approximations is found, with maximum or close to the maximum approximation size. In particular, each input from the domain should be included in at least several approximations. This is usually already achieved as a byproduct of the repetitive execution of our algorithms.

Deterministic Cover. The natural idea is to cover the full input space by t different approximations. Then, it is sufficient to enumerate all possible covers during an attack, which has a multiplicative cost of t guesses per S-box. If the considered problem (key recovery or the CICO problem) has a solution, then there will be a set of guesses agreeing with the S-box inputs from the solution, and linearization will succeed. In the ideal case, the cover becomes a partition, as in the case of the inverse mapping, and this was used for the linearization attack in [44].

In order to optimize the attack, we need to minimize t , the number of approximations in the cover. This is an instance of the NP-hard SetCover problem. This problem can be straightforwardly modeled as an integer linear programming system (MILP), see e.g. [42], yielding good quality or optimal solutions, depending on the S-box size.

However, it is inevitable that the best cover by approximations has many overlaps between the approximations. This reduces the effective approximation size $2^n/t$, even if all considered approximations have the same size VecLin_S .

Probabilistic Uniform Cover. We now propose a new alternative method, which yields a probabilistic attack but often with no or small loss in the effective approximation size. It is in particular relevant to S-boxes where approximations do not form a nice partition. The idea is again to cover the input space with approximations, but instead of minimizing their number we will assign probabilities (weights) to each of the available approximations. The requirement would be to obtain a (close to) uniform distribution on the input space, while maximizing the effective approximation size. Then, during the attack, we will sample an approximation per each S-box according to the chosen probabilities. In case of attack failure, we can simply rerun the attack, because the uniform distribution guarantees that each correct input from the solution always has a fixed chance of being covered by the chosen approximation.

More formally, consider an S-box $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ and let $X_1, \dots, X_N \subseteq \mathbb{F}_2^n$ be the intersection sets of affine approximations $A_1, \dots, A_N$ with S . Our goal is to assign probabilities $P = (p_1, \dots, p_N)$, $\sum_{i=1}^N p_i = 1$ such that for each $x \in \mathbb{F}_2^n$, the probability to be covered by an approximation sampled according to P is close to uniform, that is,

$$\left| \Pr_{X \leftarrow \overset{P_i}{X_i}} [x \in X_i] - g2^{-n} \right| \leq \epsilon 2^{-n}, \quad (1)$$

where $g = \sum_{i=1}^N p_i |X_i|$ is the expected approximation size and ϵ is a small error, e.g., $\epsilon = 0.01$. In addition, we aim to maximize g within the above constraints.

This problem can be efficiently solved by linear programming. For each $x \in \mathbb{F}_2^n$, observe that the probability to be selected is equal to $\sum_{i: x \in X_i} p_i$. It follows that the problem can be formulated as a linear programming on real variables p_i , $0 \leq p_i \leq 1$, with the 2^n constraints of the form (1) and the objective function $g = \sum_{i=1}^N p_i |X_i|$ to maximize.

Having a solution P to the problem, we can now mount an attack on a cryptographic primitive by sampling the approximation for each involved S-box application according to the distribution P . Due to the uniformity, each attack attempt is independent from the other, and every S-box call has success chance $g2^{-n}$ (up to the error $\epsilon 2^{-n}$) to be linearized. In other words, if $S(x) = y$ in the solution to the cryptographic attack, with probability $g2^{-n}$ we have $A(x) = y$, where A is the sampled approximation.

The advantage of the uniform cover method over the deterministic method is that the problem is linear in its nature in contrast to the combinatorial nature of the previous method. Therefore, with a sufficient number of approximations ($c \cdot 2^n$ for a small factor c), the 2^n -dimensional linear space of probabilities ($\Pr[x \in X_i]_{x \in \mathbb{F}_2^n}$) saturates which allows to achieve arbitrary distributions. As long as a sufficient number of approximations of large size are available, the method should succeed without further reducing the effective approximation size. A good example is given by quadratic S-boxes, where a perfect uniform cover from any single approximation is always easy to construct directly.

Proposition 7. *Let $S: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be of algebraic degree 2 and let $A: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ be its affine approximation of size $\hat{g}$. Then, it is possible to construct a uniform cover (as defined above) with $\epsilon = 0$ and expected approximation size $g = \hat{g}$.*

Proof. Using Theorem 1, we can set $\{X_i\}_i = \{(A \cap S) \oplus \Delta \mid \Delta \in \mathbb{F}_2^n\}$, $N = 2^n$, and $p_1 = \dots = p_N = 2^{-n}$. Clearly, every $x \in \mathbb{F}_2^n$ is hit by exactly $\hat{g}$ different offsets Δ (given by $\Delta = x \oplus y$ for y in $A \cap S$) and the statement follows.

Remark 2. For monomial S-boxes, one can obtain a similar result by shifting the approximation set multiplicatively by all nonzero finite field elements. However, this will not ensure uniformity at $x = 0$, which we then would need to treat in alternative ways or simply ignore the bias.

7.2 Solving the CICO Problem over SPN by Linearization

We consider a $\text{CICO}_{c,r}$ problem over an SPN structure with R S-box layers ($m = r + c$ S-boxes each) and MDS linear layers, starting and ending with an MDS layer. This is a common structure among arithmetization-oriented primitives such as Poseidon [19, 21]. Although many such primitives work over large finite fields, often prime fields, a recent trend considers application of smaller S-boxes in some layers (e.g. Monolith, [20], Skyscraper [10]). In the full version of the paper, we recall some basic tricks for skipping some S-boxes in the CICO problem, based on available degrees of freedom. Note that for SPNs starting and ending with an S-box layer, the number of attacked rounds can be directly increased by 2 rounds.

Linearization Attack. By linearizing all $A = mR - (r - c) = r(R - 1) + cR$ active S-boxes (using one of the techniques described in Subsect. 7.1), where $r - c$ correspond to the degrees of freedom used to skip some S-boxes, the problem reduces to an $\mathbb{F}_2$ -linear system of cn equations and variables which has one solution on average. However, this system does not constraint correctness of the approximations (i.e., that the actual inputs in the solution fall in the guessed approximation). Therefore, the success probability of one such attempt is given by $(g2^{-n})^A = (g2^{-n})^{mR - (r - c)}$ (where g is the expected approximation size) and the inverse of this probability determines the expected number of attempts to succeed, each costing a linear system solution over cn bits.

In particular, for a single round ($R = 1$), we have the cost of $(g2^{-n})^{-2c} = 2^{2c(n - \log_2 g)}$, while the generic brute-force attack has cost 2^{nc} . This implies that linearization attack for 1 round is effective whenever $g > 2^{n/2}$.

For a larger number of rounds, the rate size starts to contribute to the cost, since the extra degrees of freedom it provides are insufficient to counter the number of new active S-boxes. We will consider the minimal case of $r = c$ ($m = 2c$), so that the complexity is given by $(2^n/g)^{2cR}$, for example $(2^n/g)^{4c}$ for 2 rounds, $(2^n/g)^{6c}$ for 3 rounds, etc. We conclude with the following theorem.

Theorem 3. *The linearization attack applied to $\text{CICO}_{c,c}$ (i.e., $r = c$, excluding $R = 1$ where r can be any) improves over the exhaustive search attack when the average approximation size g for the S-box satisfies*

$$g > 2^{n(1-\frac{1}{2R})},$$

assuming the cost of solving the linear system is comparable to one evaluation (see Table 5). The total time complexity is then given by

$$\left(\frac{2^n}{g}\right)^{2Rc} = 2^{2Rc \cdot (n - \log_2 g)}.$$

constructions and solutions of square linear systems of dimension cn .

Remark 3. In practice, at least an extra factor of cn should be considered in the complexity cost to accommodate the construction and solution of the linearization system. Note that the matrix can be constructed by querying the cn unit vectors in the input, and the system solving should take comparable time (assuming that the evaluation cost takes a quadratic number of bit operations).

Table 5. Minimum (average) approximation size of an S-box needed for a linearization attack on $\text{CICO}_{c,c}$ (i.e., $r = c$) over SPN with MDS linear layers and m parallel n -bit S-boxes per layer.

Rounds	$g(n)$	$n = 4$	$n = 8$	$n = 16$	$n = 32$	$n = 64$
1	$2^{n/2}$	4	16	2^8	2^{16}	2^{32}
2	$2^{3n/4}$	8	64	2^{12}	2^{24}	2^{48}
3	$2^{5n/6}$	10.1	102	$2^{13.3}$	$2^{26.7}$	$2^{53.3}$
4	$2^{7n/8}$	11.3	128	2^{14}	2^{28}	2^{56}
R	$2^{n(1-\frac{1}{2R})}$					

We have experimentally verified the attack on the $\text{CICO}_{3,3}$ problem over a 1-round SPN with random linear layers and the 8-bit SKINNY S-box, which has a best affine approximation with 47 inputs. By finding a large number of (around 300) approximations in the size range between 40 and 47, we managed to build an almost-uniform probabilistic cover with average approximation size 43.7, leading to $(2^n/g) = 2^{2.55}$ and complexity $2^{5.1c}$ equal to $2^{15.3}$ linearizations for $c = 3$, compared to 2^{24} evaluations in an exhaustive search. During $2^{18.1}$ linearization attempts, we succeeded $15 = 2^{3.9}$ times, yielding the experimental expected number of attempts $2^{-14.2}$. The improvement over $2^{15.3}$ is explained by the fact that there existed two different solutions to the generated CICO problem, found equally often by our method.

8 Conclusions

In this work, we presented efficient algorithms for the linearization of S-boxes in practice for the first time. Our heuristic algorithms quickly provide strong affine approximations of a given S-box. Furthermore, for n up to 8 bits, our exhaustive algorithms can be used to prove (or, in principle, disprove and improve) the optimality of the approximation. The feasibility of the exhaustive algorithms for larger n is limited, and our heuristic results raise many interesting questions about vectorial linearity. As an application, we derived a methodology and illustrated a linearization attack against a CICO problem, which is of high relevance for permutation-based designs. We believe our powerful toolkit achieves a milestone in understanding linearization and our results lead to a wide range of open problems and research directions.

References

1. Banik, S., Barooti, K., Durak, F.B., Vaudenay, S.: Cryptanalysis of LowMC instances using single plaintext/ciphertext pair. *IACR Trans. Symm. Cryptol.* **2020**(4), 130–146 (2020). <https://doi.org/10.46586/tosc.v2020.i4.130-146>
2. Banik, S., et al.: Midori: a block cipher for low energy. In: Iwata, T., Cheon, J.H. (eds.) ASIACRYPT 2015. LNCS, vol. 9453, pp. 411–436. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48800-3_17
3. Baum, C., de Saint Guilhem, C.D., Kales, D., Orsini, E., Scholl, P., Zaverucha, G.: Banquet: short and fast signatures from AES. In: Garay, J.A. (ed.) PKC 2021. LNCS, vol. 12710, pp. 266–297. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-75245-3_11
4. Beierle, C., et al.: Schwaemm and ESCH: lightweight authenticated encryption and hashing using the Sparkle permutation family. NIST Round **2** (2019), version 1.1
5. Beierle, C., et al.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016. LNCS, vol. 9815, pp. 123–153. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53008-5_5
6. Beierle, C., Leander, G.: New instances of quadratic APN functions. *IEEE Trans. Inf. Theory* **68**(1), 670–678 (2022). <https://doi.org/10.1109/TIT.2021.3120698>
7. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Menezes, A.J., Vanstone, S.A. (eds.) CRYPTO 1990. LNCS, vol. 537, pp. 2–21. Springer, Heidelberg (1991). https://doi.org/10.1007/3-540-38424-3_1
8. Biryukov, A., Perrin, L.: On reverse-engineering S-boxes with hidden design criteria or structure. In: Gennaro, R., Robshaw, M. (eds.) CRYPTO 2015. LNCS, vol. 9215, pp. 116–140. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-47989-6_6
9. Biryukov, A., Tureček, P., Udovenko, A.: Algorithmic toolkit for linearization of S-boxes - Supporting code (2026). <https://doi.org/10.5281/zenodo.18715466>
10. Bouvier, C., et al.: Skyscraper: Fast hashing on big primes. *Cryptology ePrint Archive, Report 2025/058* (2025). <https://eprint.iacr.org/2025/058>
11. Brinkmann, M., Leander, G.: On the classification of APN functions up to dimension five. *DCC* **49**(1–3), 273–288 (2008). <https://doi.org/10.1007/s10623-008-9194-6>

12. Carlet, C.: Bounds on the nonlinearity of differentially uniform functions by means of their image set size, and on their distance to affine functions. *IEEE Trans. Inf. Theory* **67**(12), 8325–8334 (2021). <https://doi.org/10.1109/TIT.2021.3114958>
13. Carlet, C.: Two generalizations of almost perfect nonlinearity. *J. Cryptol.* **38**(2), 20 (2025). <https://doi.org/10.1007/s00145-025-09538-5>
14. Carlet, C., Charpin, P., Zinoviev, V.A.: Codes, bent functions and permutations suitable for DES-like cryptosystems. *DCC* **15**(2), 125–156 (1998). <https://doi.org/10.1023/A:1008344232130>
15. Chen, L., Fu, F.: On the nonlinearity of multi-output boolean functions. *Acta Scientificarum Naturalium University Nankaiensis* **34**(4), 28–33 (2001). https://www.alljournals.cn/view_abstract.aspx?pcid=01BA20E8BA813E1908F3698710BBFEFEE816345F465FEBA5&cid=96E6E851B5104576C2DD9FC1FBCB69EF&jid=2F663905325EBF0C8638CFC155B7BC91&aid=A4F31984B5D18D95&yid=14E7EF987E4155E6
16. Chen, S., Guo, J., List, E., Shi, D., Zhang, T.: Diving deep into the preimage security of AES-like hashing. In: Joye, M., Leander, G. (eds.) *EUROCRYPT 2024*, Part I. LNCS, vol. 14651, pp. 398–426. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-58716-0_14
17. Courtois, N.T., Amiel, F., de Fonvillars, A.B.: On maximum size simultaneous linear approximations in Ascon and Keccak and related translation and differential properties. *Cryptology ePrint Archive*, Report 2024/802 (2024). <https://eprint.iacr.org/2024/802>
18. Gorodilova, A.A., et al.: An overview of the eight international olympiad in cryptography “Non-Stop University CRYPTO”. *Sibirskie Elektronnye Matematicheskie Izvestiya [Siberian Electronic Mathematical Reports]* **19**(1), A9–A37 (2022). <https://doi.org/10.33048/semi.2022.19.023>. <http://mi.mathnet.ru/semr1488>
19. Grassi, L., et al.: Poseidon(2)b: binary field versions of poseidon/poseidon2. *IACR Commun. Cryptol.* **2**(4) (2026). <https://doi.org/10.62056/a66ce0zn4>
20. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schafnegger, M., Walch, R.: Monolith: circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. *IACR Trans. Symmetric Cryptol.* **2024**(3), 44–83 (2024). <https://doi.org/10.46586/tosc.v2024.i3.44-83>
21. Grassi, L., Khovratovich, D., Schafnegger, M.: Poseidon2: a faster version of the poseidon hash function. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) *AFRICACRYPT 23*. LNCS, vol. 14064, pp. 177–203. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-37679-5_8
22. Guido, B., Joan, D., Michaël, P., Gilles, V.: Cryptographic sponge functions. 2011-STMicroelectronics NXP Semiconductors, Version 0.1 January 14 (2011)
23. Guo, J., Liao, G., Liu, G., Liu, M., Qiao, K., Song, L.: Practical collision attacks against round-reduced SHA-3. *J. Cryptol.* **33**(1), 228–270 (2020). <https://doi.org/10.1007/s00145-019-09313-3>
24. Guo, J., Peyrin, T., Poschmann, A., Robshaw, M.J.B.: The LED block cipher. In: Preneel, B., Takagi, T. (eds.) *CHES 2011*. LNCS, vol. 6917, pp. 326–341. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-23951-9_22
25. Kazymyrov, O., Kazymyrova, V., Oliynykov, R.: A method for generation of high-nonlinear S-boxes based on gradient descent. *Cryptology ePrint Archive*, Paper 2013/578 (2013). <https://eprint.iacr.org/2013/578>
26. Kim, S., et al.: AIM: symmetric primitive for shorter signatures with stronger security. In: *ACM CCS 2023*, pp. 401–415. ACM, New York, NY, USA (2023). <https://doi.org/10.1145/3576915.3616579>

27. Langevin, P.: Classification of APN cubics in dimension 6 over $\text{GF}(2)$ (2012). <http://langevin.univ-tln.fr/project/apn-6/apn6.html>
28. Liu, J., Mesnager, S., Chen, L.: On the nonlinearity of S-boxes and linear codes. *Cryptogr. Commun.* **9**(3), 345–361 (2017). <https://doi.org/10.1007/S12095-015-0176-Z>
29. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Helleseht, T. (ed.) EUROCRYPT'93. LNCS, vol. 765, pp. 386–397. Springer, Heidelberg (1994). https://doi.org/10.1007/3-540-48285-7_33
30. Matsui, M.: New block encryption algorithm MISTY. In: Biham, E. (ed.) FSE'97. LNCS, vol. 1267, pp. 54–68. Springer, Heidelberg (1997). <https://doi.org/10.1007/BFb0052334>
31. Matsui, M., Yamagishi, A.: A new method for known plaintext attack of FEAL cipher. In: Rueppel, R.A. (ed.) EUROCRYPT'92. LNCS, vol. 658, pp. 81–91. Springer, Heidelberg (1993). https://doi.org/10.1007/3-540-47555-9_7
32. Nagy, G.P.: On the minimum Hamming distance between vectorial Boolean and affine functions. *Cryptogr. Commun.* **17**(6), 1703–1720 (2025). <https://doi.org/10.1007/s12095-025-00808-4>
33. Nagy, G.P.: Sidon sets, thin sets, and the nonlinearity of vectorial Boolean functions. *J. Comb. Theory Ser. A* **212**, 106001 (2025). <https://doi.org/10.1016/j.jcta.2024.106001>
34. Nyberg, K.: On the construction of highly nonlinear permutations. In: Rueppel, R.A. (ed.) EUROCRYPT'92. LNCS, vol. 658, pp. 92–98. Springer, Heidelberg (1993). https://doi.org/10.1007/3-540-47555-9_8
35. Nyberg, K.: Differentially uniform mappings for cryptography. In: Helleseht, T. (ed.) EUROCRYPT'93. LNCS, vol. 765, pp. 55–64. Springer, Heidelberg (1994). https://doi.org/10.1007/3-540-48285-7_6
36. Nyberg, K., Knudsen, L.R.: Provable security against differential cryptanalysis. In: Brickell, E.F. (ed.) Advances in Cryptology – CRYPTO'92. LNCS, vol. 740, pp. 566–574. Springer, Heidelberg (1993). https://doi.org/10.1007/3-540-48071-4_41
37. Perrin, L., Udovenko, A.: Exponential s-boxes: a link between the s-boxes of BelT and Kuznyechik/Streebog. *IACR Trans. Symm. Cryptol.* **2016**(2), 99–124 (2016). <https://doi.org/10.13154/tosc.v2016.i2.99-124>. <https://tosc.iacr.org/index.php/ToSC/article/view/567>
38. Perrin, L., Wiemer, F., Stennes, L.: S-boxes used in cryptographic schemes (2019). <https://doc.sagemath.org/html/en/reference/cryptography/sage/crypto/sboxes.html>, SageMath module
39. Qiao, K., Song, L., Liu, M., Guo, J.: New collision attacks on round-reduced Keccak. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part III. LNCS, vol. 10212, pp. 216–243. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56617-7_8
40. Ryabov, V.: Nonlinearity of APN functions: comparative analysis and estimates. [*Prikladnaya Diskretnaya Matematika*] (61), 15–27 (2023). <https://doi.org/10.17223/20710410/61/2>. <https://www.mathnet.ru/rus/pdm810>
41. Sage Developers: SageMath, the Sage Mathematics Software System (Version 10.7) (2025). <https://www.sagemath.org>
42. Sasaki, Yu., Todo, Y.: New algorithm for modeling S-box in MILP based differential and division trail search. In: Farshim, P., Simion, E. (eds.) SecITC 2017. LNCS, vol. 10543, pp. 150–165. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-69284-5_11

- 43. Zaverucha, G., et al.: Picnic. Technical report, National Institute of Standards and Technology (2017). <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-1-submissions>
- 44. Zhang, K., Wang, Q., Yu, Y., Guo, C., Cui, H.: Algebraic attacks on round-reduced rain and full AIM-III. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part III. LNCS, vol. 14440, pp. 285–310. Springer, Singapore (2023). https://doi.org/10.1007/978-981-99-8727-6_10